Algorithm Mid-Term Suggestion(HSC 24B)

Topics

- **Binary Search Fundamentals**: Understand why the array **must** be sorted and how the search space is halved in each step .
- **Time & Space Complexity**: Memorize the Best, Average, and Worst-case time complexities for all taught algorithms & difference between them.
- **Sorting Comparison**: Differentiate between **Bubble**, **Selection**, and **Insertion** Sort based on their "Adaptive" nature and the total number of **swaps** performed .
- **Design Strategies**: Explain how **Divide and Conquer** works in Merge Sort. Also explain the concept of divide, conquer and combine.
- **Stability & In-place Sorting**: Identify which algorithms preserve the order of duplicate elements (Stability) also which algorithms preform in-place sorting. And what do you mean by stability and in-place sorting.
- **Non-Comparison Sorting**: Explain the mechanics of **Counting Sort** and why it is limited to small integer ranges .
- Algorithm Definition, Properties, asymptotic notations, Algorithm design strategies.
- Pseudo-code: Binary Search, Quick Sort, Bubble Sort.
- Scenario Based Time complexity analysis: for e.g. in which case insertion sort works the best, when selection sort works best, what is quick sort worst case etc.

Demo Preparation Questions

Set 1: Search Logic & Elementary Sorting

1. (a) Explain how the "Divide by Half" logic of Binary Search leads to a logarithmic time complexity.

1. (b) Use the **Insertion Sort** algorithm to sort the array `[7, 3, 5, 2]`. Show the state of the array after the first **two** elements are inserted into their correct positions .

Set 2: Complexity Analysis & Tracing

2. (a) Examine the "Adaptive" property of Bubble Sort. How does adding a "swapped" flag change its **Best Case** time complexity?

2. (b) Use the **Selection Sort** algorithm to sort the array `[10, 2, 8, 4]`. Provide the array state after **Pass 1** and **Pass 2**, and **Calculate** the total swaps made .

Set 3: Paradigms & Constraints

3. (a) Explain why **Merge Sort** is considered a "Divide and Conquer" algorithm while **Counting Sort** is considered a "Non-Comparison" algorithm .

3. (b) Evaluate a scenario where you are sorting a database of 1,000,000 items on a server with **unlimited RAM** but a requirement for **Stability**. Which would you choose between Quick Sort and Merge Sort? Justify your choice .

3.(c) You have a mobile device with limited RAM. Between merge sort vs Quick sort which one is suitable.

Quick-Review

Algorithm	Best Case	Worst Case	Space Complexity	Stability	In-place
Binary Search	O(1)	O(log n)	O(1) (Iterative)	N/A	Yes
Bubble Sort	O(n) (Optimized)	O(n^2)	O(1)	Yes	Yes
Selection Sort	O(n^2)	O(n^2)	O(1)	No	Yes
Insertion Sort	O(n)	O(n^2)	O(1)	Yes	Yes
Merge Sort	O(n log n)	O(n log n)	O(n)	Yes	No
Quick Sort	O(n log n)	O(n^2)	O(log n)	No	Yes
Counting Sort	O(n+k)	O(n+k)	O(n+k)	Yes	No